

BLM-0482 — NESNELERİN İNTERNETİ

IoT Simülasyon Projesi

MQTT Tabanlı Sensör Veri Toplama ve Görselleştirme Sistemi

Takım 7 · 2025–2026 Bahar Dönemi

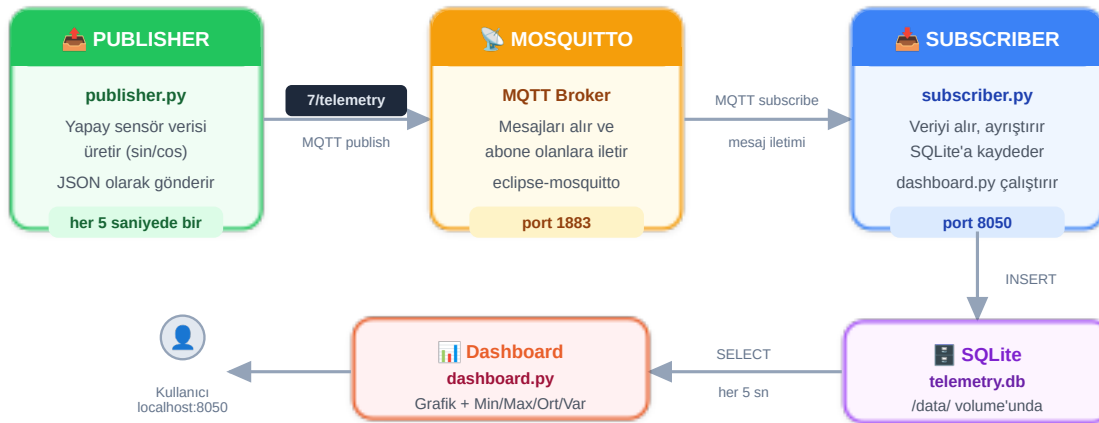
1. Projeye Genel Bakış

Bu proje, gerçek bir IoT (Nesnelerin İnterneti) sistemini simüle etmektedir. Sisteme göre yapay sensör verileri (sıcaklık, nem, ışık) üretilmekte, MQTT protokolü üzerinden iletilmekte, veritabanına kaydedilmekte ve canlı bir web arayüzünde grafiksel olarak sunulmaktadır.

Kullanılan MQTT Topic: 7/telemetry — Takım numarası/telemetry formatında tanımlanmıştır.

2. Sistem Mimarisi

Sistem üç ana bileşenden oluşmakta ve Docker ile paketlenmektedir:



3. Proje Dosya Yapısı

simulation/

— docker-compose.yml	← Tüm servisleri bir arada yöneten ana yapılandırma
— Taskfile.yml	← Kolay komut çalıştırma aracı (task up, task down...)
— mosquitto/	
— mosquitto.conf	← MQTT broker yapılandırması
— publisher/	
— publisher.py	← Yapay sensör verisi üreten ve gönderen yazılım
— requirements.txt	← Python bağımlılıkları
— Dockerfile	← Publisher container tarifi
— subscriber/	
— subscriber.py	← MQTT'yi dinleyip veritabanına kaydeden yazılım
— dashboard.py	← Web tabanlı görselleştirme paneli
— requirements.txt	← Python bağımlılıkları
— Dockerfile	← Subscriber container tarifi

4. Dosyaların Detaylı Açıklaması



docker-compose.yml

Servis orkestrasyon dosyası

Projedeki tüm servislerin (Mosquitto, Publisher, Subscriber) tek bir komutla nasıl başlatılacağını tanımlar. Docker Compose bu dosyayı okuyarak her servisi ayrı bir container olarak çalıştırır ve aralarındaki ağ bağlantısını otomatik kurar.

```
services:
  mosquitto:
    image: eclipse-mosquitto:latest # MQTT broker servisi
    ports:
      - "1883:1883" # Dış dünya → container port yönlendirmesi
    volumes:
      - ./mosquitto/mosquitto.conf:... # Yapılandırma dosyası mount edilir

  publisher:
    build: ./publisher # publisher/Dockerfile'dan imaj derlenir
    depends_on:
      - mosquitto # Mosquitto hazır olmadan başlamaz
    restart: on-failure # Hata alırsa otomatik yeniden başlar

  subscriber:
    build: ./subscriber
    depends_on:
      - mosquitto
    ports:
      - "8050:8050" # Dashboard bu porttan erişilir
    volumes:
      - telemetry_data:/data # Veritabanı kalıcı volume'da saklanır
    restart: on-failure

volumes:
  telemetry_data: # SQLite dosyasının saklandığı kalıcı alan
```

depends_on Servis başlatma sırasını belirler. Publisher ve Subscriber, Mosquitto ayağa kalkmadan çalışmaz.

restart: on-failure Broker henüz hazır değilken bağlantı hatası alınabilir. Bu ayar servisi

otomatik yeniden başlatır.

volumes

Container silinse bile telemetry veritabanı kaybolmaz; kalıcı Docker volume'unda tutulur.



mosquitto/mosquitto.conf

MQTT broker yapılandırması

Eclipse Mosquitto MQTT broker'ının nasıl çalışacağını belirler. Broker, publisher'dan gelen mesajları alıp subscriber'a ileten merkezi dağıtım noktasıdır.

```
listener 1883      # MQTT standardı port 1883'te dinleme yapılır
allow_anonymous true # Kimlik doğrulaması gerektirmez (simülasyon için)
```

listener 1883

MQTT protokolünün standart portu 1883'tür. Publisher ve Subscriber bu porta bağlanır.

allow_anonymous

Gerçek üretimdeki sistemlerde kullanıcı adı/şifre gerekir. Simülasyonda kolaylık için anonim bağlantıya izin verilmiştir.



Ödevin 1. aşamasını karşılar. Her 5 saniyede bir yapay sensör verisi üretir ve bu veriyi JSON formatında MQTT broker'ına gönderir. Veriler sinüs/kosinüs fonksiyonları ile gerçekçi dalgalanmalar gösterir.

```
BROKER = "mosquitto" # Docker ağında broker'ın adı
PORT   = 1883
TOPIC  = "7/telemetry" # Takım 7'nin topic adresi
INTERVAL = 5           # Saniye cinsinden yayın aralığı

t = 0
while True:
    # Sinüs dalgasıyla gerçekçi veri + rastgele gürültü
    sicaklik = round(22 + 5 * math.sin(t * 0.1) + random.uniform(-0.5, 0.5), 2)
    nem       = round(55 + 10 * math.cos(t * 0.08) + random.uniform(-1, 1), 2)
    isik      = round(400 + 150 * math.sin(t * 0.05) + random.uniform(-10, 10), 2)

    payload = {
        "sensor_id": "temp_01",
        "values": {"sicaklik": sicaklik, "nem": nem, "isik": isik},
        "unit": "metric",
        "timestamp": datetime.now(timezone.utc).isoformat()
    }
    client.publish(TOPIC, json.dumps(payload))
    time.sleep(INTERVAL)
```

paho-mqtt

Python için MQTT istemci kütüphanesidir. `client.publish()` ile mesaj gönderilir.

math.sin / math.cos

Verinin zamanla dalgalanmasını sağlar. Düz sabit veri yerine gerçek sensörlere benzer sinyal üretir.

random.uniform

Küçük rastgele gürültü ekler. Grafiklerde doğal titreme görünmesini sağlar.

json.dumps(payload)

Veri sözlüğünü JSON string'e çevirir. MQTT metin tabanlı çalıştığı için bu dönüşüm gereklidir.

timestamp (UTC)

Her mesaja ISO 8601 formatında zaman damgası eklenir. Zaman serisi grafiği için kritiktir.

Örnek üretilen JSON verisi:

```
{"sensor_id": "temp_01", "values": {"sicaklik": 24.5, "nem": 63.2,
"isik": 487.1}, "unit": "metric", "timestamp": "2026-04-29T09:58:00Z"}
```

**subscriber/subscriber.py**

Veri alıcı ve veritabanı yazıcı — MQTT Subscriber

Ödevin 3a ve 3b maddelerini karşılar. 7/telemetry topic'ini sürekli dinler; gelen her JSON mesajını ayrıştırır ve SQLite veritabanına kaydeder.

```
DB_PATH = "/data/telemetry.db" # Docker volume içindeki kalıcı dosya

def on_message(client, userdata, msg):
    data = json.loads(msg.payload) # JSON string → Python dict
    v    = data["values"]
    conn.execute(
        "INSERT INTO telemetry (sensor_id, sicaklik, nem, isik, timestamp)"
        " VALUES (?, ?, ?, ?, ?)",
        (data["sensor_id"], v["sicaklik"], v["nem"], v["isik"], data["timestamp"])
    )

client.on_message = on_message
client.connect(BROKER, PORT, 60)
client.subscribe(TOPIC)
client.loop_forever() # Sonsuz dinleme döngüsü
```

on_message callback MQTT'den her mesaj geldiğinde otomatik çağrılan fonksiyondur. İçinde veritabanı yazma işlemi gerçekleşir.

SQLite Sunucu gerektirmeyen hafif bir veritabanıdır. Tüm veriler tek bir .db dosyasında saklanır.

threading.Lock MQTT mesajları ayrı bir iş parçacığında gelir. Lock, veritabanına eş zamanlı yazımı önleyerek veri bütünlüğünü korur.

loop_forever() Program sonlanmadan sürekli mesaj bekler. Sistemi sürekli aktif tutar.

/data/telemetry.db Docker volume'a mount edilmiş klasördedir. Container silinse bile veriler kaybolmaz.

Veritabanı Tablo Yapısı

Sütun	Tür	Açıklama
id	INTEGER	Otomatik artan birincil anahtar
sensor_id	TEXT	Sensör kimliği (ör. "temp_01")
sicaklik	REAL	Sıcaklık değeri (°C)
nem	REAL	Nem değeri (%)
isik	REAL	Işık miktarı (lx)
timestamp	TEXT	ISO 8601 zaman damgası



Ödevin 3c ve 3d maddelerini karşılar. Plotly Dash framework'ü ile geliştirilmiş web tabanlı bir paneldir. Her 5 saniyede veritabanını okuyarak grafikleri ve istatistikleri otomatik günceller. İki farklı tema sunar: klasik açık tema ve Grafana'dan esinlenilen koyu tema.

Özellikler

Zaman Serisi Grafikleri	Her sensör için ayrı çizgi grafik. X eksenini zaman, Y eksenini ölçüm değeridir.
İstatistik Paneli	Her grafiğin yanında Min, Max, Ortalama ve Varyans değerleri canlı hesaplanır.
Otomatik Güncelleme	dcc.Interval bileşeni 5 saniyede bir callback tetikler, sayfa yenilenmeden veriler güncellenir.
İki Tema	Klasik (beyaz) ve Grafana (koyu) tema arasında anlık geçiş yapılabilir.
LIVE Göstergesi	Grafana temasında yeşil yanıp sönen LIVE badge sistemin aktif olduğunu gösterir.

Kullanılan Kütüphaneler

Kütüphane	Görev
Dash	Web uygulama framework'ü, bileşen sistemi ve callback mekanizması
Plotly	İnteraktif grafik motoru
Pandas	SQLite'tan okunan veriyi tablo olarak işleme
SQLite3	Veritabanı okuma

İstatistik Hesaplamaları

Değer	Hesaplama	Anlamı
Min	series.min()	O ana kadar ölçülen en düşük değer
Max	series.max()	O ana kadar ölçülen en yüksek değer
Ort	series.mean()	Tüm ölçümlerin aritmetik ortalaması
Var	series.var()	Değerlerin ortalamadan ne kadar saptığının ölçüsü

**Taskfile.yml**

Geliştirici komut yöneticisi

Makefile'in modern YAML tabanlı alternatifidir. Uzun Docker komutlarını kısa ve akılda kalıcı task adlarına dönüştürür.

Komut	Ne yapar
task up	Tüm servisleri build edip arka planda başlatır
task down	Tüm servisleri durdurur ve container'ları kaldırır
task restart	down ardından up çalıştırır
task logs	Tüm servislerin log akışını canlı gösterir
task logs:publisher	Sadece publisher loglarını gösterir
task logs:subscriber	Sadece subscriber loglarını gösterir
task ps	Çalışan container'ları listeler
task build	Cache'siz, sıfırdan imaj derler
task clean	Container ve kalıcı volume'ları tamamen siler

5. Projenin Çalıştırılması

Gereksinimler

Docker	Container çalıştırmak için gerekli. <code>docker --version</code> ile kontrol edilir.
Docker Compose	Çoklu container yönetimi. Modern Docker kurulumlarında dahili gelir.
Task CLI	Taskfile komutları için. <code>sh -c "\$(curl -fsSL https://taskfile.dev/install.sh)" -- -d -b /usr/local/bin</code> ile kurulur.

Başlatma Adımları

- 1 Proje klasörüne gir: `cd simulation/`
- 2 Tüm servisleri başlat: `task up`
- 3 Logları izle (isteğe bağlı): `task logs`
- 4 Tarayıcıda dashboard'u aç: `http://localhost:8050`
- 5 Sistemi durdurmak için: `task down`

6. Veri Akışı — Adım Adım

- 1 **Publisher başlar** — Mosquitto broker'a bağlanır ve her 5 saniyede veri üretmeye başlar.
- 2 **JSON mesajı oluşturulur** — Sıcaklık, nem, ışık değerleri hesaplanır; `sensor_id` ve `timestamp` eklenerek JSON'a dönüştürülür.
- 3 **MQTT ile gönderilir** — Mesaj `7/telemetry` topic'ine publish edilir.
- 4 **Mosquitto broker iletir** — Bu topic'e abone olan tüm subscriber'lara mesajı gönderir.
- 5 **Subscriber alır ve kaydeder** — `on_message` callback'i tetiklenir; JSON ayrıştırılır ve SQLite'a INSERT yapılır.
- 6 **Dashboard okur ve gösterir** — Her 5 saniyede tüm veritabanı okunur; grafikler ve istatistikler hesaplanarak sayfaya yansıtılır.

7. Kullanılan Teknolojiler

Teknoloji	Kategori	Kullanım Amacı
MQTT	Protokol	IoT cihazlar arası hafif mesajlaşma protokolü
Eclipse Mosquitto	Broker	Açık kaynaklı MQTT broker — mesajları yönlendirir
Python / paho-mqtt	Dil / Kütüphane	Publisher ve Subscriber yazılımı
SQLite	Veritabanı	Telemetri verilerinin kalıcı depolanması
Plotly Dash	Web Framework	İnteraktif web dashboard
Docker / Compose	Container	Servislerin izole ve taşınabilir çalıştırılması
Taskfile	Araç	Geliştirici komut yönetimi